

Лабораторная работа № 1.3

«Объектно-ориентированный лексический анализатор»

23 марта 2026 г.

Ольга Ивенкова, ИУ9-62Б

Цель работы

Целью данной работы является приобретение навыка реализации лексического анализатора на объектно-ориентированном языке без применения каких-либо средств автоматизации решения задачи лексического анализа.

Индивидуальный вариант

Идентификаторы: последовательности латинских букв и цифр, начинающиеся с буквы.

Строковые константы — последовательности строковых секций, записанных слитно. Строковые секции: либо последовательность символов, ограниченных апострофами, апостроф внутри строки описывается как два апострофа подряд, не пересекают границы строк текста, либо знак «#», за которым следует десятичная константа (код символа).

Пример строковой константы: «'hello'#10#13'world'» (эта строковая константа состоит из 4 строковых секций, однако является единым токеном).

Лексический домен для защиты

Знаки отношений: >, >=, <, <=

Реализация

Файл `LexerAnalyse.java`

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
```

```

import java.nio.file.Files;
import java.nio.file.Path;
import java.util.*;

class Position implements Comparable<Position> {
    private final String text;
    private int index;
    private int line;
    private int col;

    public Position(String text) {
        this(text, 0, 1, 1);
    }

    private Position(String text, int index, int line, int col) {
        this.text = text;
        this.index = index;
        this.line = line;
        this.col = col;
    }

    public boolean isEOF() {
        return index >= text.length();
    }

    public int getIndex() {
        return index;
    }

    public int getLine() {
        return line;
    }

    public int getCol() {
        return col;
    }

    public int getCp() {
        return isEOF() ? -1 : text.codePointAt(index);
    }

    public int peekCp(int offset) {
        int i = index;
        while (offset-- > 0) {
            if (i >= text.length()) {
                return -1;
            }
        }
    }
}

```

```

    }
    int cp = text.codePointAt(i);
    i += Character.charCount(cp);
}
return i >= text.length() ? -1 : text.codePointAt(i);
}

public void next() {
    if (isEOF()) {
        return;
    }

    int cp = text.codePointAt(index);
    int count = Character.charCount(cp);

    if (cp == '\r') {
        index += count;
        if (index < text.length() && text.charAt(index) == '\n') {
            index++;
        }
        line++;
        col = 1;
    } else if (cp == '\n') {
        index += count;
        line++;
        col = 1;
    } else {
        index += count;
        col++;
    }
}

public Position copy() {
    return new Position(text, index, line, col);
}

@Override
public int compareTo(Position o) {
    return Integer.compare(this.index, o.index);
}

@Override
public String toString() {
    return "(" + line + ", " + col + ")";
}
}

```

```

class Fragment {
    public final Position start;
    public final Position follow;

    public Fragment(Position start, Position follow) {
        this.start = start;
        this.follow = follow;
    }

    @Override
    public String toString() {
        return start + "-" + follow;
    }
}

enum DomainTag {
    IDENT,
    STRING,
    LESS,
    LESS_EQUAL,
    GREATER,
    GREATER_EQUAL,
    END_OF_PROGRAM
}

abstract class Token {
    public final DomainTag tag;
    public final Fragment coords;

    protected Token(DomainTag tag, Position start, Position follow) {
        this.tag = tag;
        this.coords = new Fragment(start, follow);
    }

    @Override
    public abstract String toString();
}

class IdentToken extends Token {
    public final int code;

    public IdentToken(int code, Position start, Position follow) {
        super(DomainTag.IDENT, start, follow);
        this.code = code;
    }
}

```

```

        @Override
        public String toString() {
            return tag + " " + coords + ": " + code;
        }
    }

    class StringToken extends Token {
        public final String value;

        public StringToken(String value, Position start, Position follow) {
            super(DomainTag.STRING, start, follow);
            this.value = value;
        }

        @Override
        public String toString() {
            return tag + " " + coords + ": " + value;
        }
    }

    class SpecToken extends Token {
        public SpecToken(DomainTag tag, Position start, Position follow) {
            super(tag, start, follow);
        }

        @Override
        public String toString() {
            return tag + " " + coords + ":";
        }
    }

    class Compiler {
        private final Map<String, Integer> nameToCode = new HashMap<>();
        private final List<String> names = new ArrayList<>();
        private final List<Message> messages = new ArrayList<>();

        public int addName(String name) {
            return nameToCode.computeIfAbsent(name, k -> {
                int code = names.size();
                names.add(name);
                return code;
            });
        }

        public String getName(int code) {

```

```

        return names.get(code);
    }

    public void addError(Position pos, String text) {
        messages.add(new Message(pos.copy(), text, true));
    }

    public void addWarning(Position pos, String text) {
        messages.add(new Message(pos.copy(), text, false));
    }

    public List<Message> getMessages() {
        messages.sort(Comparator.comparingInt(m -> m.pos.getIndex()));
        return messages;
    }

    static class Message {
        public final Position pos;
        public final String text;
        public final boolean isError;

        Message(Position pos, String text, boolean isError) {
            this.pos = pos;
            this.text = text;
            this.isError = isError;
        }
    }
}

class Scanner {
    public final String program;
    private final Compiler compiler;
    private final Position cur;

    public Scanner(String program, Compiler compiler) {
        this.program = program;
        this.compiler = compiler;
        this.cur = new Position(program);
    }

    private static boolean isLatinLetter(int cp) {
        return (cp >= 'A' && cp <= 'Z') || (cp >= 'a' && cp <= 'z');
    }

    private static boolean isLatinLetterOrDigit(int cp) {
        return isLatinLetter(cp) || (cp >= '0' && cp <= '9');
    }
}

```

```

}

private static boolean isDigit(int cp) {
    return cp >= '0' && cp <= '9';
}

private void skipWhitespace() {
    while (!cur.isEOF() && Character.isWhitespace(cur.getCp())) {
        cur.next();
    }
}

private Token scanIdentifier() {
    Position start = cur.copy();
    cur.next();

    while (!cur.isEOF() && isLatinLetterOrDigit(cur.getCp())) {
        cur.next();
    }

    String name = program.substring(start.getIndex(), cur.getIndex());
    return new IdentToken(compiler.addName(name), start, cur.copy());
}

private Token scanStringConstant() {
    Position start = cur.copy();
    StringBuilder value = new StringBuilder();

    while (true) {
        int cp = cur.getCp();

        if (cp == '\\') {
            cur.next();

            while (true) {
                cp = cur.getCp();

                if (cp == -1) {
                    compiler.addError(cur.copy(), "end of program found, '\\' expected");
                    return new StringToken(value.toString(), start, cur.copy());
                }

                if (cp == '\\n' || cp == '\\r') {
                    compiler.addError(cur.copy(), "newline in string constant");
                    cur.next();
                    return new StringToken(value.toString(), start, cur.copy());
                }
            }
        }
    }
}

```

```

    }

    if (cp == '\\') {
        cur.next();
        if (cur.getCp() == '\\') {
            value.append('\\');
            cur.next();
            continue;
        }
        break;
    }

    value.appendCodePoint(cp);
    cur.next();
}
} else if (cp == '#') {
    if (!isDigit(cur.peekCp(1))) {
        break;
    }

    cur.next(); // #
    long code = 0;
    boolean overflow = false;
    int digits = 0;

    while (isDigit(cur.getCp())) {
        int d = cur.getCp() - '0';
        if (code > (Character.MAX_CODE_POINT - d) / 10L) {
            overflow = true;
        } else {
            code = code * 10 + d;
        }
        digits++;
        cur.next();
    }

    if (digits == 0) {
        compiler.addError(cur.copy(), "digit expected after #");
        return new StringTokenizer(value.toString(), start, cur.copy());
    }

    if (overflow || code > Character.MAX_CODE_POINT) {
        compiler.addError(cur.copy(), "invalid character code");
        value.append('\uFFFD');
    } else {
        value.appendCodePoint((int) code);
    }
}

```



```

    }
    } else {
        break;
    }

    int next = cur.getCp();
    if (!(next == '\\' || (next == '#' && isDigit(cur.peekCp(1))))) {
        break;
    }
}

return new StringToken(value.toString(), start, cur.copy());
}

public Token nextToken() {
    while (true) {
        skipWhitespace();

        if (cur.isEOF()) {
            return new SpecToken(DomainTag.END_OF_PROGRAM, cur.copy(), cur.copy());
        }

        int cp = cur.getCp();

        if (isLatinLetter(cp)) {
            return scanIdentifier();
        }

        // Знаки отношений: >, >=, <, <=
        if (cur.getCp() == '<') {
            Position start = cur.copy();
            cur.next();
            if (cur.getCp() == '=') {
                cur.next();
                return new SpecToken(DomainTag.LESS_EQUAL, start, cur.copy());
            } else {
                return new SpecToken(DomainTag.LESS, start, cur.copy());
            }
        }

        if (cur.getCp() == '>') {
            Position start = cur.copy();
            cur.next();
            if (cur.getCp() == '=') {
                cur.next();

```

```

        return new SpecToken(DomainTag.GREATER_EQUAL, start, cur.copy());
    } else {
        return new SpecToken(DomainTag.GREATER, start, cur.copy());
    }
}

if (cp == '\\') {
    return scanStringConstant();
}

if (cp == '#') {
    if (isDigit(cur.peekCp(1))) {
        return scanStringConstant();
    }
    compiler.addError(cur.copy(), "unexpected character: #");
    cur.next();
    continue;
}

compiler.addError(cur.copy(),
"unexpected character: " + new String(Character.toChars(cp)));
cur.next();
}
}

}

public class LexerAnalyse {
    public static void main(String[] args) {
        try {
            String text = Files.readString(Path.of(args[0]), StandardCharsets.UTF_8);
            Compiler compiler = new Compiler();
            Scanner scanner = new Scanner(text, compiler);

            List<Token> tokens = new ArrayList<>();
            Token tok;
            do {
                tok = scanner.nextToken();
                tokens.add(tok);
            } while (tok.tag != DomainTag.END_OF_PROGRAM);

            for (Token t : tokens) {
                System.out.println(t);
            }

            for (Compiler.Message msg : compiler.getMessages()) {

```

```

        System.out.println((msg.isError ? "Error" : "Warning") +
            " " + msg.pos + ": " + msg.text);
    }
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

Тестирование

Входные данные

```

var123
123abc
''
'hello'#10#11'world'
'@'
<=
>

```

Вывод на stdout

```

IDENT (1, 1)-(1, 7): 0
IDENT (2, 4)-(2, 7): 1
STRING (3, 1)-(3, 3):
STRING (4, 1)-(4, 21): hello
σworld
STRING (5, 1)-(5, 4): @
LESS_EQUAL (6, 1)-(6, 3):
GREATER (7, 1)-(7, 2):
END_OF_PROGRAM (7, 2)-(7, 2):
Error (2, 1): unexpected character: 1
Error (2, 2): unexpected character: 2
Error (2, 3): unexpected character: 3

```

Вывод

Приобрела навык реализации лексического анализатора на объектно-ориентированном языке без применения каких-либо средств автоматизации решения задачи лексического анализа.